

DATA PIPELINE

DATASCIENCETECHINSTITUTE

JABRI MEHDI

BEN YAHIA SAMI

SAHLI MONGI SELIM

HALLEB MONCEF

2025/2026

Contents :

I - Introduction

II - Work Distribution

III - Working environment, tools & online services

IV - Model

4.1 Structure and naming conventions

4.2 Identifier convention

4.3 Type reuse

4.4 Integrity constraints

4.5 Limitations

V - Scenarios

5.1 From Guest to Member: Tracking Player Transitions

5.2 Members with at least 3 years of seniority who have won a tournament

5.3 Members categorized by age group: Minor, Adult, and Senior

5.4 Total training hours completed per coach

5.5 Facilities ranked by total usage hours this year (Complex solution)

5.6 Members count over the past 5 years

5.7 Membership Growth Report

5.8 Training groups report

5.9 Tournament analytics

5.10 Facility bookings

VI - AI use

I – Introduction

This project was carried out as part of the Data Pipeline course and focuses on the design and implementation of a data management platform for a tennis club, the RG Tennis Club.

The goal is to provide the club with a centralized and structured system capable of handling all aspects of its daily operations, including member registrations, coaching staff, training sessions, facility bookings, equipment tracking, and tournament management.

II – Work Distribution

Member	Contributions
Sami ~32.5%	• Acted as the team coordinator, assigning tasks to each member and ensuring overall project organization • Converted the agreed-upon club structure into an XML schema (with modifications) • Wrote visualization scenarios 1, 4, and 5 • Ensured all scenarios were functional within a unified environment • Populated the database with entries to enrich scenario outputs
Mehdi ~27.5%	• Designed and wrote the database • Wrote visualization scenarios 2, 3, and 6 • Structured and organized the PDF report
Selim ~17.5%	• Imagined and wrote one exploitation scenario (XML output) • Imagined and wrote one exploitation scenario (JSON output) • Wrote the Python program to load, parse, validate an XML file against the schema, and apply an XSL stylesheet
Moncef ~22.5%	• Imagined and wrote one exploitation scenario (XML output) • Imagined and wrote one exploitation scenario (JSON output) • Wrote the Java program to load, parse, validate an XML file against the schema, and apply an XSL stylesheet • Provided a second implementation of one scenario (Python or Java)

III – Working environment, tools & online services

a. Working environment

- Visual studio code
- Notepad++
- JDK

b. Tools

- PYTHON
- lxml
- JAVA
- XSLT Transformation
- XML Schema validation

c. Online services

- xslttransform.net
- BaseX
- <https://developer.mozilla.org/fr/docs/Web/CSS>
- https://www.w3schools.com/Html/html_css.asp

IV – Model

Our model represents a tennis club and contains multiple entities such as **members**, **coaches**, **facilities**, and more.

4.1 Structure and naming conventions

Every element is written in uppercase, while complex types are written in Pascal Case (like *Information*, *TrainingGroup*) to make them easily distinguishable at first glance.

TENNIS_CLUB is the root element, composed of container elements such as **MEMBERS**, which in turn hold occurrences of their respective single elements such as **MEMBER**. This structure allows any section irrelevant to the current task to collapse for readability purposes.

One exception is **MEMBERSHIPS**, which was designed as a child of **MEMBER** rather than a direct child of the root. Had it been placed at the root level, each membership would have required a reference back to its parent member. In a real-world scenario where members accumulate a large number of expired memberships, retrieving the memberships of a specific member would have required a stylesheet traversal for what

should be easily done. Nesting **MEMBERSHIPS** directly under **MEMBER** creates a direct link between the two without the need for any ID or XML transformation.

4.2 Identifier convention

Every **@Id** attribute is the identifier of its own element, while child elements referencing another entity's identifier (such as **FACILITYID** or **COACHID**) are treated as foreign identifiers. This makes it immediately clear whether an identifier belongs to the current element or references an external one.

4.3 Type reuse

Shared simple and complex types, such as **Level**, **Information**, and **Contact**, were defined once and reused across multiple elements. This reduces redundancy and ensures that any modification to one of these types is automatically reflected everywhere it is used, without requiring changes to each element.

4.4 Integrity constraints

Keys were declared on the root element to enforce the uniqueness of every **@Id** attribute across the document. Corresponding keyrefs were defined for each use of a foreign identifier, ensuring that every reference points to an existing element and preventing broken references caused by typos.

4.5 Limitations

Despite these advantages, the model has one critical limitation: certain fields cannot be protected from invalid user input. For instance, nothing in the schema prevents a **DATE_HOUR_BEGIN** from being set later than its corresponding **DATE_HOUR_END**. Similarly, in the tournament section, a member of an inappropriate level could have their identifier incorrectly entered in the **MEMBERSID** section, and the **WINNER** element, which stores the winner's first and last name as a free-text string, offers no guarantee that the value entered actually corresponds to a participant. To our knowledge, XML Schema 1.0 does not natively support this kind of security.

V – Scenarios

5.1 From Guest to Member: Tracking Player Transitions

- This scenario identifies guest players who, after booking a facility to play at the club, subsequently registered as official members.

5.2 Members with at least 3 years of seniority who have won a tournament

- This scenario identifies members with at least 3 years of seniority who have also won at least one tournament.

5.3 Members categorized by age group: Minor, Adult, and Senior

- This scenario retrieves the member IDs of players categorized by age group: Minor, Adult, and Senior (age 60 and above).

5.4 Total training hours completed per coach

- This scenario computes the total number of hours each coach has spent leading training sessions.

5.5 Facilities ranked by total usage hours this year (Complex solution)

- Similar to Scenario 4, this scenario calculates the cumulative usage time of each facility. Unlike Scenario 4, however, it accounts for two distinct sources of usage: time spent hosting training sessions, as well as time allocated to member and guest bookings for individual matches.

This scenario proved particularly complex as it required computing the difference between two DateTime elements to obtain a duration, then aggregating these durations to produce a total. Two limitations of XSLT 1.0 made this difficult. First, the sum() function does not accept expressions as input, ruling out any computation directly passed as an argument. Second, since variables in XSLT are immutable, it is not possible to declare a variable and increment it across iterations to accumulate a total.

The solution was to implement a recursive template. On each call, the template extracts the first element of the relevant XPath, computes its duration, and then calls itself with that element excluded from the node-set and an accumulator parameter incremented

accordingly. This accumulator would, in the end, be the sum of all the durations, allowing the total duration to be built up without relying on sum().

This scenario was slightly more complex than Scenario 4, as the recursive template had to handle two distinct XPath node-sets simultaneously and sum their durations within each call. An additional case had to be considered for the situation where one of the two node-sets is exhausted before the other to make sure the remaining elements of the non-empty node-set continue to be computed until completion.

A secondary challenge concerned the time computation itself. An initial approach was handled separately in the case where the end minutes were lower than the beginning minutes, resulting in poor readability and two different computations.

The final solution converts both timestamps into a total number of minutes (multiplying hours by 60 and adding minutes), then subtracts the beginning value from the end value. In other words, we were computing the difference between the end and beginning times. This approach is more readable, more concise, and naturally handles the case where the end minutes are lower than the beginning minutes without requiring separate logic.

5.6 Members count over the past 5 years

- This scenario retrieves the total number of members registered each year over the past 5 years.

5.7 Membership Growth Report

- This scenario creates a JSON-like summary for all club members.

It shows the member ID, full name, number of memberships, and the last membership level.

It also determines whether the member is active or expired based on the end date of the latest membership.

5.8 Training groups report

- This scenario generates an XML analytics report for all training groups in the tennis club.

It counts the number of members in each group and the number of training sessions linked to it.

It also classifies the activity level as Low, Medium, or High based on the number of sessions.

5.9 Tournament analytics

– This scenario generates an XML report for all tournaments in the club.

It displays the tournament ID, name, level, number of participants, and prize.

It also indicates whether the tournament has a winner and lists the winner if present.

5.10 Facility bookings

– This scenario produces a JSON-like report for each facility in the club.

It counts how many bookings are associated with each facility.

It also assigns a usage category (Low, Medium, or High) depending on the booking count.

VI – AI use

AI was used to populate the database once its structure and initial elements had been correctly established. We consider that generating fictional elements such as names, dates, and phone numbers does not constitute an intellectual task, nor does it contribute to a deeper understanding of the XML language. The focus of our work remained on the design of the schema, the modeling choices, and the development of the stylesheets.

Prompt:

“How do I test and run the project?”

Result:

Run the Python script from the project folder with the virtual environment activated.

Then check that the XML loads, validates against the XSD, and the transformation creates result.xml.

Open the output file to confirm the result is correct.

Prompt:

“Adapt the code to the new XML schema.”

Result:

Update all XPath paths and element names in the XSLT so they match the new XML schema exactly.

XML is case-sensitive, so tags like TENNIS_CLUB and TRAINING_GROUPS must be used exactly as written.

If the schema changes, the transformation logic must be updated too.

Prompt:

“What should I change if the schema changes?”

Result:

Change every XPath that points to renamed elements, attributes, or new nested structures.

Also update any count, filter, or reference logic that depends on old field names. If needed, adjust the Python or Java code only for file names or parsing rules.

Prompt:

“Explain why the output is empty.”

Result:

The output is empty because the XPath paths do not match the XML structure or the case of the tags.

A wrong root name, wrong attribute name, or wrong element name will return no data. The transformation may run correctly but still produce blank values if the paths are wrong.

Prompt:

“How do I install lxml inside my virtual environment?”

Result:

Activate the virtual environment first, then run `python -m pip install lxml`. This installs the package in the correct environment used by your script. Then test it with `python -c "from lxml import etree; print('ok')"`.

“What command should I use to compile and run my Java XML program?”

Compile with `javac YourProgram.java`.

Run with `Java YourProgram`.

Make sure the XML, XSD, and XSL files are in the same folder or use the correct paths.

Prompt:

“I want through my VS Code terminal to execute the scenarios with BaseX and generate the output directly without going through the file with the code.”

Result:

```
"C:\Users\hp\Downloads\BaseX123\basex\bin\basex.bat" -o
"C:\Users\hp\Downloads\DataPipelineProject\scenario1_output.html" -q
"xslt:transform(doc('file:///C:/Users/hp/Downloads/DataPipelineProject/tennis_club.xml'),
doc('file:///C:/Users/hp/Downloads/DataPipelineProject/scenario1_en.xsl'))"
```

Prompt:

"The minor and senior cutoff dates are defined in a static way. I want them to be dynamic based on today's date."

Result:

```
<!-- y5 = current year -->
<xsl:variable name="y5" select="string(number($currentYear))"/>
<!-- y4 = current year minus 1 -->
<xsl:variable name="y4" select="string(number($currentYear) - 1)"/>
<!-- y3 = current year minus 2 -->
<xsl:variable name="y3" select="string(number($currentYear) - 2)"/>
<!-- y2 = current year minus 3 -->
<xsl:variable name="y2" select="string(number($currentYear) - 3)"/>
<!-- y1 = current year minus 4 -->
<xsl:variable name="y1" select="string(number($currentYear) - 4)"/>
```

Prompt:

"Je souhaite obtenir des connaissances afin de pouvoir faire des stylesheets. Je souhaite donc que tu m'aides de la manière que tu jugera le plus approprié (document qui répertorie les parties importantes par exemple ou autre si cest mieux) Pour me permettre d'apprendre de manière efficace J'ai plus ou moins la logique sur les values of, for each. Je n'ai pas de connaissance poussé sur les lignes qui "embellissent" le rendu du stylesheet Prend le temps nécessaire pour me répondre."

Result:

Voilà un guide interactif — chaque carte se déplie pour afficher l'explication et l'exemple de code. Voici une lecture rapide de ce que j'ai couvert et pourquoi :

Structure & déclarations — les 4 éléments qui "configurent" le stylesheet avant tout traitement : xsl:stylesheet, xsl:output, xsl:param, xsl:key. Ce sont des déclarations globales, pas du traitement à proprement parler.

Logique & contrôle de flux — xsl:variable, xsl:choose/when/otherwise et xsl:sort. Tu maîtrises déjà for-each et value-of, donc j'ai centré ces cartes sur les subtilités (RTF vs node-set pour les variables, position de xsl:sort, etc.).

XPath — les parties que tu connais moins probablement : l'astuce translate() pour comparer des dates (il n'y a pas de type date en XSLT 1.0), l'échappement < obligatoire dans les attributs XML, et les fonctions natives les plus utilisées.

HTML + CSS inlined — c'est la partie "embellissement" que tu mentionnes. Le principe est simple : le CSS est écrit dans un <style> statique dans le <head>, et les balises HTML avec leurs classes sont écrites directement dans le XSL — seul le contenu variable passe par xsl:value-of. xsl:text sert à insérer des espaces ou de la ponctuation entre deux instructions XSL.

Si tu veux, on peut aller plus loin sur un point précis (les templates nommés, apply-templates, ou comment structurer une feuille plus complexe).

Prompt:

"sum(//TRAINING_SESSION[@Id = \$id]/ (number(substring(DATE_HOUR_END,12,2))) - (number(substring(DATE_HOUR_BEGIN,12,2))) -

(number(substring(DATE_HOUR_END,15,2)) <
number(substring(DATE_HOUR_BEGIN,15,2)))) cette syntaxe fonctionne t-elle ? “

Result:

“Non.

Pourquoi ça ne fonctionne pas

sum() en XPath/XSLT 1.0 ne peut pas prendre une expression arithmétique directement dans les crochets [] ou sur un node-set de cette façon.

Problèmes précis :

- sum() attend une liste de nœuds ou valeurs simples, pas une expression complexe par nœud
- tu fais un calcul “inline” sur chaque nœud → pas supporté en XSLT 1.0 standard
- sum(//TRAINING_SESSION[...] / (expression)) ❌ invalide